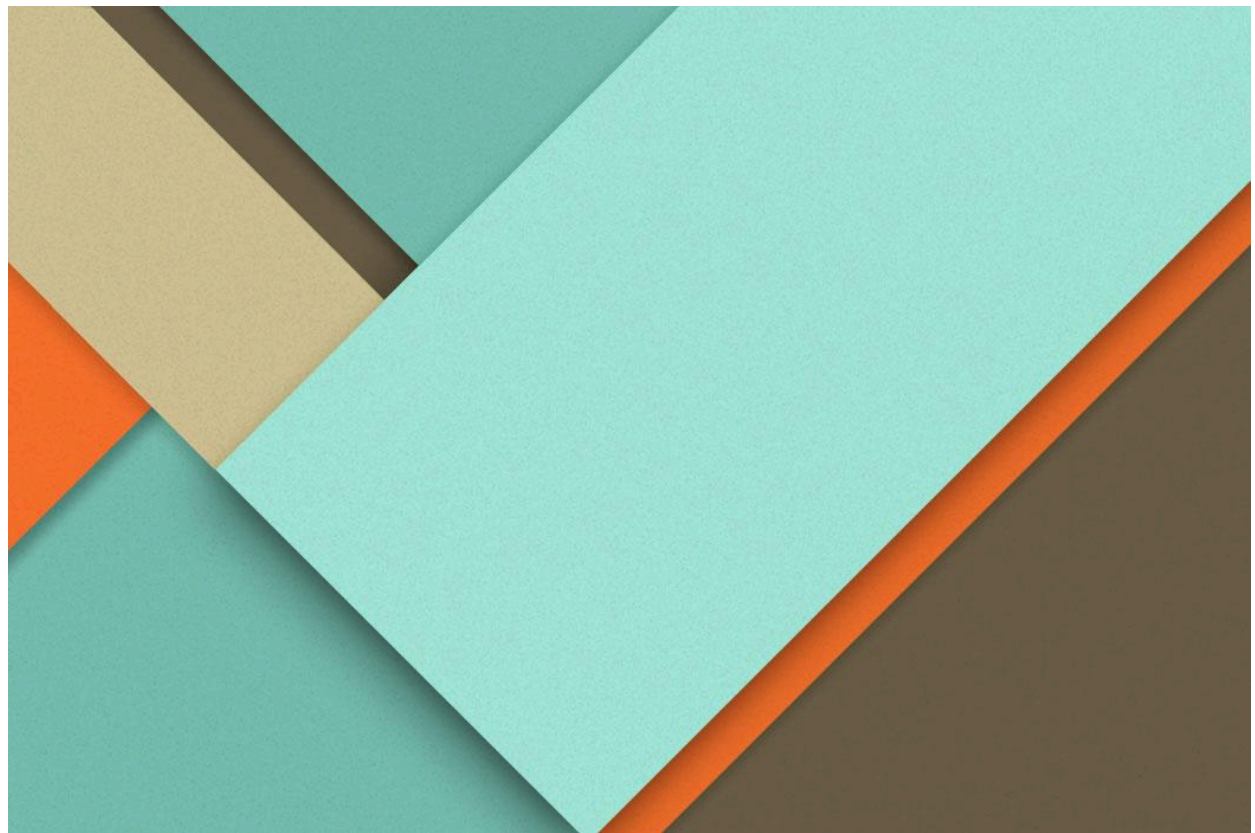




Proyecto ATP

07.06.2024

Jesus David Gelves Cajiao

30000098650

jdgelvesc@correo.usbcali.edu.co

Técnicas Avanzadas de Programación



Problema-Requisitos-Restricciones

Los requerimientos que aplican para cada uno de los módulos son las siguientes:

Client

- Requerimientos funcionales
 - o Ofrece una interacción/interfaz en modo CLI para la selección del problema a resolver, y la cantidad de valores a procesar.
 - o Imprime y/o salva a disco los resultados obtenidos para el problema seleccionado, acorde con solicitud del usuario.
 - o Permite hacer la petición de apagado (i.e., shutdown) de todo el sistema, empleando algún mecanismo de autenticación.
- Restricciones tecnológicas:
 - o Está implementado en Golang.
 - o Interopera mediante sockets con ProblemSolver, empleando formato JSON para intercambio de datos.

ProblemSolver

- Requerimientos funcionales
 - o Consume datos de DataServer para ser empleados en el problema a resolver.
 - o Resuelve el problema solicitado por Client.
 - o Lleva un log de las operaciones realizadas.
- Restricciones tecnológicas:
 - o Está implementado en Python.
 - o Interopera mediante HTTP con DataServer, empleando formato JSON para intercambio de datos.
 - o Emplea de manera explícita y coordinada algún mecanismo que brinde confidencialidad a los datos en tránsito con DataServer (independiente del protocolo de transmisión).

o Aplica patrones de diseño creacionales en la instanciación del problema a resolver.

DataSet

- Requerimientos funcionales

o Genera de manera aleatoria y empleando diferentes distribuciones de probabilidad una cantidad

específica de números enteros positivos en un rango también aleatorio.,

o Sirve los datos generados en respuesta a una petición HTTP, empleando el formato JSON.

- Restricciones tecnológicas:

o Está implementado en Flask-Python.

Emplea de manera explícita y coordinada algún mecanismo que brinde confidencialidad a los

datos en tránsito con ProblemSolver (independiente del protocolo de transmisión).

o Aplica inversión de dependencias en la generación de los números aleatorios.

o Está en la capacidad de emplear las distribuciones normal y uniforme.

Problemas: los problemas que resuelve ProblemSolver son los siguientes.

- FizzBuzz: especificado previamente en un taller.

- FibonacciVerifier: especificado previamente en un taller.

- PrimeClassifier: Clasifica un número, en primo, semiprimo-cuadrático, semiprimo, o ninguno de los

anteriores. Un número semiprimo es aquel que tiene exactamente sólo dos divisores primos diferentes. Si

los dos divisores primos son iguales, el número es un semiprimo-cuadrático.

APLICANDO EL TDD

Empezando por el módulo de cliente_project empezamos por el cliente_project/root.go

```

1 package cmd
2
3 import (
4     "bytes"
5     "cliente_project/util"
6     "encoding/json"
7     "fmt"
8     "io"
9     "log"
10    "os"
11
12    "github.com/spf13/cobra"
13 )
14
15 type Data struct {
16     Problem      string
17     MinNumber    int64
18     MaxNumber    int64
19     CountNumbers int64
20     OutPutFile   bool
21     OutPutCmd    bool
22     Kill         bool
23 }
24
25 type Result struct {
26     Numbers []int
27     Results []string
28 }
29
30 var problem string
31 var minNumber int64
32 var maxNumber int64
33 var countNumbers int64
34 var outPutFile bool
35 var outPutCmd bool
36
37 var rootCmd = &cobra.Command{
38     Use:   "cliente_project",
39     Short: "Client for solving problems",
40     Long:  `Client for solving problems like FizzBuzz, FibonacciVerifier, PrimeClassifier.
41     Example usage: go run main.go --problem=FizzBuzz --min=20 --max=100 --count=50 --output_file or --output_cmd`,
42     Run: func(cmd *cobra.Command, args []string) error {
43         data := Data{
44             Problem:      problem,
45             CountNumbers: countNumbers,
46             MinNumber:    minNumber,
47             MaxNumber:    maxNumber,
48             OutPutFile:   outPutFile,
49             OutPutCmd:    outPutCmd,
50             Kill:         false,
51         }
52         if !outPutFile && !outPutCmd {
53             outPutCmd = true
54         }
55     },
56 }

```

```

37 var rootCmd = &cobra.Command{
76     return fmt.Errorf("error receiving data: %v", err)
77 }
78
79 buffer := make([]byte, 4096)
80 var result Result
81 var jsonDataBytes bytes.Buffer
82
83 for {
84     n, err := reader.Read(buffer)
85     if err != nil {
86         if err == io.EOF {
87             break
88         }
89         fmt.Println("Error reading from socket:", err)
90     }
91     jsonDataBytes.Write(buffer[:n])
92 }
93
94 err = json.Unmarshal(jsonDataBytes.Bytes(), &result)
95 if err != nil {
96     fmt.Println("error")
97 }
98
99 if outPutFile {
100     util.WriteResultToFile(result.Numbers, result.Results, problem)
101 }
102
103 if outPutCmd {
104     for i, number := range result.Numbers {
105         fmt.Printf("%d %s\n", number, result.Results[i])
106     }
107 }
108
109 return nil
110 },
111 },
112 },
113 },
114 },
115 },
116
117 func Execute() {
118     if err := rootCmd.Execute(); err != nil {
119         os.Exit(1)
120     }
121 }
122
123 func init() {
124     rootCmd.Flags().StringVarP(&problem, "problem", "p", "FizzBuzz", "Problem to be resolved")
125     rootCmd.Flags().Int64VarP(&minNumber, "min", "m", 0, "Minimum number")
126     rootCmd.Flags().Int64VarP(&maxNumber, "max", "M", 100, "Maximum number")
127     rootCmd.Flags().Int64VarP(&countNumbers, "count", "c", 10, "Count of numbers")
128     rootCmd.Flags().BoolVarP(&outPutFile, "output_file", "o", false, "Output file")
129     rootCmd.Flags().BoolVarP(&outPutCmd, "output_cmd", "t", false, "Output cmd")
130 }
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

- El código define las estructuras data y result para almacenar información sobre el problema y los resultados.
- Se declara variables para almacenar los parámetros del problema (tipo de problema, mínimo, máximo, cantidad de números, salida a archivo y salida por consola).
- Se configura el comando principal que es el rootCmd con su descripción, uso y ejemplo.

2.Explicación del comando mencionado:

- La función RunE se ejecuta cuando se invoca el comando, se crea una instancia de la estructura data con los valores de las variables y si no se especifica el formato de salida, se aplica el uso de la flag e imprime en consola por defecto. Después, se convierten los datos del problema a formato JSON.
- Se establece una conexión con el servidor utilizando la función ConnectToServer.
- Se envían los datos JSON al servidor mediante la función SendData.
- Se recibe la respuesta del servidor como un flujo de datos usando la función ReceiveData.
- Se convierte la respuesta completa en formato JSON y se almacena en la estructura Result.

3. Salida:

- Se verifica si se ha solicitado la salida a archivo.
- Si es así, se utiliza la función WriteResultToFile para escribir los números y resultados a un archivo.
- Se verifica si se ha solicitado la salida por consola.
- Si es así, se imprime cada número junto con su resultado correspondiente.

shutdown.go

```

1  package cmd
2
3  import (
4      "cliente_project/util"
5      "encoding/json"
6      "fmt"
7      "log"
8      "strings"
9
10     "github.com/spf13/cobra"
11 )
12
13 var inputPassword string
14
15 var shutdownCmd = &cobra.Command{
16     Use: "shutdown",
17     Short: "Shutdown the server",
18     Long: `Shutdown the server by providing the correct password`,
19     RunE: func(cmd *cobra.Command, args []string) error {
20         fmt.Println("Starting shutdown...")
21
22         if inputPassword == "" {
23             return fmt.Errorf("a password is required, use -p 'password'")
24         }
25
26         actualPasswords := util.Read()
27         if len(actualPasswords) == 0 {
28             return fmt.Errorf("the system doesn't have a set password")
29         }
30
31         hashes := strings.Split(actualPasswords, ",")
32         if util.CheckPasswordHash(inputPassword, hashes) {
33             fmt.Println("Password is correct.")
34         } else {
35             fmt.Println("Password is incorrect.")
36         }
37
38         if !util.CheckPasswordHash(inputPassword, hashes) {
39             return fmt.Errorf("the provided password is incorrect")
40         }
41
42         data := Data{
43             Kill: true,
44         }
45
46         jsonData, err := json.Marshal(data)
47         if err != nil {
48             log.Fatalf("error: %v", err)
49         }
50
51         conn, err := util.ConnectToServer("localhost:65535")
52         if err != nil {
53             return fmt.Errorf("error connecting to server: %v", err)
54         }
55         defer conn.Close()
56
57         err = util.SendData(jsonData, conn)
58         if err != nil {
59             return fmt.Errorf("error sending data: %v", err)
60         }
61     },
62 }

```

- Se declara una variable inputPassword para almacenar la contraseña ingresada por el usuario.

- Se define el subcomando shutdownCmd con su descripción, uso y explicación detallada.
- La función RunE se ejecuta cuando se invoca el subcomando shutdown.
- Se imprime un mensaje indicando el inicio del apagado.
- Se verifica si se ha proporcionado una contraseña a través de la variable inputPassword.
- Si no se ha proporcionado contraseña, se muestra un error indicando que es necesaria.
- Se lee la contraseña real del servidor usando la función util.Read (esta función no se muestra pero se asume que lee la contraseña almacenada de forma segura).
- Si no se encuentra una contraseña almacenada, se muestra un error indicando que el sistema no tiene contraseña configurada.
- Se divide la cadena de contraseñas almacenadas en una lista usando la función strings.Split (asumiendo que las contraseñas están separadas por comas).
- Se llama a la función util.CheckPasswordHash (no mostrada) para verificar si la contraseña ingresada coincide con alguna de las contraseñas almacenadas.
- Si la contraseña es incorrecta, se muestra un mensaje y se devuelve un error.
- Si la contraseña es correcta, se crea una instancia de la estructura Data con el campo Kill establecido en true.
- Se convierte la estructura Data a formato JSON.
- Se establece una conexión con el servidor utilizando la función ConnectToServer.
- Se envían los datos JSON al servidor mediante la función SendData.
- La función init agrega el subcomando shutdownCmd al comando raíz (rootCmd) definido en otro archivo (posiblemente main.go).
- Dentro del subcomando, se define un parámetro -p o --password para que el usuario pueda especificar la contraseña al ejecutar el comando.

files.go

```

1  package util
2
3  import (
4      "bufio"
5      "fmt"
6      "os"
7      "strconv"
8  )
9
10 func Write(hashedPassword string) {
11     file, err := os.OpenFile("files/password.txt", os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0644)
12     if err != nil {
13         fmt.Println("error creating or opening file:", err)
14         return
15     }
16     defer file.Close()
17
18     _, err = fmt.Fprintf(file, "%s\n", hashedPassword)
19     if err != nil {
20         fmt.Println("error writing to the file:", err)
21         return
22     }
23 }
24
25 func Read() string {
26     file, err := os.Open("files/password.txt")
27     if err != nil {
28         fmt.Println("error opening the file:", err)
29         return ""
30     }
31     defer file.Close()
32
33     scanner := bufio.NewScanner(file)
34     scanner.Scan()
35     return scanner.Text()
36 }
37
38 func WriteResultToFile(numbers []int, results []string, problem string) {
39     file, err := os.Create("files/results.txt")
40     if err != nil {
41         fmt.Println("Error creating file:", err)
42         return
43     }
44     defer file.Close()
45
46     writer := bufio.NewWriter(file)
47     writer.WriteString("Problem to solve: " + problem + "\n")
48     for i := 0; i < len(numbers); i++ {
49         str := strconv.Itoa(numbers[i])
50         writer.WriteString(str + " " + results[i] + "\n")
51     }
52     writer.Flush()
53 }
54

```

1. Write(hashedPassword string):

- Esta función almacena una contraseña hasheada (encriptada) en un archivo de texto.
- Abre el archivo "files/password.txt" en modo append (añadir al final), create (crear si no existe) y write only (solo escritura).
- Si hay un error al abrir el archivo, lo imprime y regresa sin hacer nada.
- Convierte la contraseña hasheada a un string y la escribe en el archivo junto a un salto de línea (\n).
- Si hay un error al escribir en el archivo, lo imprime y regresa sin hacer nada.

2. Read() string:

- Esta función lee la contraseña hasheada almacenada en un archivo de texto.
- Abre el archivo "files/password.txt" en modo lectura.
- Si hay un error al abrir el archivo, lo imprime y regresa una cadena vacía.
- Crea un escáner para leer el archivo línea por línea.
- Lee la primera línea del archivo.
- Si hay alguna línea leída, la regresa como cadena. Si no hay líneas o hay un error al leer, regresa una cadena vacía.

3. WriteResultToFile(numbers []int, results []string, problem string):

- Esta función escribe los resultados de un problema a un archivo de texto.
- Crea un archivo de texto llamado "files/results.txt".
- Si hay un error al crear el archivo, lo imprime y regresa sin hacer nada.
- Crea un escritor con buffer para escribir en el archivo de forma más eficiente.
- Escribe la línea "Problema a resolver: " seguido del nombre del problema.
- Recorre las listas de números y resultados.
- Convierte cada número a cadena de texto utilizando la función strconv.Itoa.
- Escribe cada línea con el formato "número resultado\n".
- vacía el buffer del escritor para asegurar que todos los datos se guardan en el archivo.

hashGenerator.go

```

File Edit Selection View Go Run Terminal Help

EXPLORER
PROJECTO APT
  cliente_project
    cmd
      root.go
      shutdown.go
    files
      password.txt
      results.txt
    util
      files.go
      hashgenerator.go
      Sockets.go
      go.mod
      go.sum
      main.go
    data_server
      Crypter
      Diagram
      distributions
      my_app.py
      env
    problem_solver
      __pycache__
      Crypter
      __pycache__
      crypter.py
      Diagram

cliente_project > util > hashgenerator.go > ...
1  package util
2
3  import (
4      "crypto/sha256"
5      "encoding/hex"
6  )
7
8  func HashPassword(password string) string {
9      hasher := sha256.New()
10     hasher.Write([]byte(password))
11     return hex.EncodeToString(hasher.Sum(nil))
12 }
13
14 func CheckPasswordHash(password string, hashes []string) bool {
15     hashedPassword := HashPassword(password)
16     for _, hash := range hashes {
17         if hash == hashedPassword {
18             return true
19         }
20     }
21     return false
22 }
23

```

1. HashPOassword:

- Esta función Hashea la contraseña que se le ingresa en Shutdown.go

2. CheckPasswordHash:

- Esta función verifica si la contraseña hasheada es la misma la que se le ingreso.

Sockets.go

```

EXPLORER
PROJECTO APT
  cliente_project
    cmd
      root.go
      shutdown.go
    files
      password.txt
      results.txt
    util
      files.go
      hashgenerator.go
      Sockets.go
      go.mod
      go.sum
      main.go
    data_server
      Crypter
      Diagram
      distributions
      my_app.py
      env
    problem_solver
      _pycache_
      Crypter
      _pycache_
      crypter.py
      Diagram
      files
      problemstatus.l...
      problem_factory_...
      client.py
      log.py
      Sockets.py

cliente_project > util > Sockets.go > SendData
1  package util
2
3  import (
4      "fmt"
5      "net"
6  )
7
8  func ConnectToServer(address string) (net.Conn, error) {
9      conn, err := net.Dial("tcp", address)
10     if err != nil {
11         return nil, fmt.Errorf("error connecting to server: %v", err)
12     }
13     return conn, nil
14 }
15
16 func SendData(data []byte, connection net.Conn) error {
17     _, err := connection.Write(data)
18     if err != nil {
19         return fmt.Errorf("error sending data: %v", err)
20     }
21     fmt.Println("Data sent to server:", string(data))
22     return nil
23 }
24
25 func ReceiveData(socketPort string) (net.Conn, error) {
26     reader, err := net.Listen("tcp", socketPort)
27
28     if err != nil {
29         return nil, err
30     }
31
32     conn, err := reader.Accept()
33     if err != nil {
34         return nil, err
35     }
36     return conn, nil
37 }

```

1. ConnectToServer(address string) (net.Conn, error):

- Esta función establece una conexión con un servidor.
- Recibe la dirección del servidor como cadena de texto (por ejemplo, "localhost:65535").
- Utiliza la función net.Dial para establecer una conexión TCP con la dirección especificada.

- Si hay un error al establecer la conexión, regresa nil para la conexión y un error detallado.
- Si la conexión se establece exitosamente, regresa la conexión y nil para el error.

2. SendData(data []byte, connection net.Conn) error:

- Esta función envía datos a un servidor ya conectado.
- Recibe los datos a enviar como un arreglo de bytes y la conexión previamente establecida.
- Utiliza la función connection.Write para escribir los datos en la conexión.
- Si hay un error al escribir los datos, regresa un error detallado.
- Si los datos se envían exitosamente, imprime un mensaje indicando el contenido enviado (convertido a cadena) y regresa nil para el error.

3. ReceiveData(socketPort string) (net.Conn, error):

- Esta función espera y recibe datos de un cliente en un puerto específico.
- Recibe el puerto de escucha como cadena de texto (por ejemplo, "localhost:62000").
- Utiliza la función net.Listen para crear un listener TCP en el puerto especificado.
- Si hay un error al crear el listener, regresa nil para la conexión y el error.
- Utiliza el método Accept del listener para esperar a que un cliente se conecte.
- Si hay un error al aceptar la conexión, regresa nil para la conexión y el error.
- Si un cliente se conecta exitosamente, regresa la conexión del cliente y nil para el error.

Problem_solver/client.py

The image shows a VS Code editor window with a project named 'PROJECTO APT'. The Explorer sidebar on the left shows the project structure, including folders like 'cliente_project', 'cmd', 'files', 'util', 'data_server', 'env', 'problem_solver', and 'files'. The file 'client.py' is selected in the Explorer and is also the active file in the editor.

The editor displays the code for 'client.py', which is a Python script for a client application. The code includes imports for 'json', 'requests', 'sys', 'logging', 'log', and 'socket'. It also imports 'listen_socket' and 'send_to_cli' from 'Sockets', and 'EncryptionHandler' from 'Crypter.crypter'. Additionally, it imports 'CreatorFibonacciVerifier', 'CreatorFizzBuzz', and 'CreatorPrimeClassifier' from 'problem_factory_method.creator_fibonacciVerifier', 'problem_factory_method.creator_fizzbuzz', and 'problem_factory_method.creator_primeClassifier' respectively.

The main function of the script is 'process_and_send_data()', which is defined as follows:

```
def process_and_send_data():
    crypter = EncryptionHandler()
    crypter.generate_keys()

    public_key_pem = crypter.public_key_pem.decode()
    url_key = "http://127.0.0.1:5000/public_key"
    data = {'public_key': public_key_pem}

    try:
        request = requests.post(url_key, json=data, timeout=10)
        request.raise_for_status()
    except requests.exceptions.RequestException as e:
        logging.error(f"Failed to send public key: {e}")
        sys.exit()

    if request.status_code == 200:
        try:
            public_key = request.json().get('public_key')
            crypter.public_key_pem = public_key
        except (ValueError, KeyError) as e:
            logging.error(f"Failed to parse public key response: {e}")
            sys.exit()
    else:
        logging.error(f"Failed to get public key, status code: {request.status_code}")
        sys.exit()

    url_numbers = "http://127.0.0.1:5000/numbers"

    while True:
        try:
            json_data = listen_socket()
            logging.info("Received JSON data from socket: %s", json_data)
        except Exception as e:
            logging.error(f"Error listening to socket: {e}")
            continue

        if 'Kill' in json_data and json_data['Kill'] is True:
            url_shutdown = "http://127.0.0.1:5000/shutdown"
            try:
                response = requests.get(url_shutdown)
                response.raise_for_status()
            except requests.exceptions.RequestException as e:
                logging.info("Shutting down...")
            sys.exit()
```

```

EXPLORER
PROJECTO APT
  cliente_project
    cmd
      root.go
      shutdown.go
    files
      password.txt
      results.txt
    util
      files.go
      hashgenerator.go
      Sockets.go
      go.mod
      go.sum
      main.go
    data_server
      Crypter
      Diagram
      distributions
      my_app.py
    env
    problem_solver
      __pycache__
      Crypter
      __pycache__
      crypter.py
      Diagram
      files
      problemstatus.l...
      problem_factory_...
      client.py
      log.py
      Sockets.py

problem_solver > client.py > process_and_send_data
13 def process_and_send_data():
14     response.raise_for_status()
15 except requests.exceptions.RequestException as e:
16     logging.info("Shutting down...")
17     sys.exit()
18
19 if 'Problem' in json_data:
20     try:
21         problem_solver = json_data['Problem']
22         count = json_data['CountNumbers']
23         min = json_data['MinNumber']
24         max = json_data['MaxNumber']
25         output_file = json_data['OutPutFile']
26         output_cmd = json_data['OutPutCmd']
27
28         data = {
29             'count': count,
30             'min': min,
31             'max': max
32         }
33
34         crypt_data = crypter.encrypt(data)
35
36         try:
37             response = requests.post(url_numbers, json=crypt_data, timeout=10)
38             response.raise_for_status()
39             logging.info("Sent encrypted data to server")
40         except requests.exceptions.RequestException as e:
41             logging.error(f"Failed to create data: {e}")
42             continue
43
44         if response.status_code == 201:
45             try:
46                 results = response.json()['encrypt_result']
47                 decrypt_results = crypter.decrypt(results)
48                 numbers = decrypt_results['numbers']
49             except (ValueError, KeyError) as e:
50                 logging.error(f"Failed to parse or decrypt results: {e}")
51                 continue
52
53             if numbers:
54                 if problem_solver == 'FizzBuzz':
55                     solver = CreatorFizzBuzz().factory_method()
56                 elif problem_solver == 'FibonacciVerifier':
57                     solver = CreatorFibonacciVerifier().factory_method()
58                 elif problem_solver == 'PrimeClassifier':
59                     solver = CreatorPrimeClassifier().factory_method()
60
61                 results = solver.solve_problem(numbers)
62
63                 result_data = {
64                     'problem_solver': problem_solver,
65                     'numbers': numbers,
66                     'results': results,
67                     'output_file': output_file,
68                     'output_cmd': output_cmd
69                 }
70
71             try:

```

The screenshot shows a VS Code editor with a project named 'PROJECTO APT'. The Explorer sidebar on the left shows the project structure, including folders like 'cliente_project', 'cmd', 'files', 'util', 'data_server', 'env', 'problem_solver', and 'Cryper'. The main editor area displays the code for 'client.py', which is part of a 'problem_solver' module. The code defines a 'process_and_send_data()' function that sends data to a server and handles various exceptions. The code is as follows:

```

13 def process_and_send_data():
76     response = requests.post(url_numbers, json=crypt_data, timeout=10)
77     response.raise_for_status()
78     logging.info("Sent encrypted data to server")
79     except requests.exceptions.RequestException as e:
80         logging.error(f"Failed to create data: {e}")
81         continue
82
83     if response.status_code == 201:
84         try:
85             results = response.json()['encrypt_result']
86             decrypt_results = crypter.decrypt(results)
87             numbers = decrypt_results['numbers']
88         except (ValueError, KeyError) as e:
89             logging.error(f"Failed to parse or decrypt results: {e}")
90             continue
91
92     if numbers:
93         if problem_solver == 'FizzBuzz':
94             solver = CreatorFizzBuzz().factory_method()
95         elif problem_solver == 'FibonacciVerifier':
96             solver = CreatorFibonacciVerifier().factory_method()
97         elif problem_solver == 'PrimeClassifier':
98             solver = CreatorPrimeClassifier().factory_method()
99
100     results = solver.solve_problem(numbers)
101
102     result_data = {
103         'problem_solver': problem_solver,
104         'numbers': numbers,
105         'results': results,
106         'output_file': output_file,
107         'output_cmd': output_cmd
108     }
109
110     try:
111         send_to_cli(result_data)
112         logging.info("Sent results to CLI")
113     except Exception as e:
114         logging.error(f"Failed to send results to CLI: {e}")
115
116     else:
117         logging.warning('No data found.')
118
119     else:
120         logging.error(f"Failed to create data, status code: {response.status_code}")
121
122     except KeyError as e:
123         logging.error(f"Missing key in JSON data: {e}")
124
125 if __name__ == "__main__":
126     try:
127         process_and_send_data()
128     except Exception as e:
129         logging.critical(f"Unhandled exception: {e}")
130         sys.exit()

```

1. Intercambio de claves:

- Crea una instancia de la clase EncryptionHandler para manejar el cifrado de datos.
- Genera un par de claves pública/privada para el cifrado.
- Convierte la clave pública a formato cadena y la envía al servidor en la dirección `http://127.0.0.1:5000/public_key` mediante una petición POST con la librería `requests`.
- Si la petición falla, se registra un error y se finaliza el programa.
- Si la petición es exitosa y el código de estado es 200, se espera recibir la clave pública del servidor en la respuesta.
- Se actualiza la clave pública del servidor en el objeto `crypter` para poder descifrar las respuestas.
- Si hay algún error al obtener o procesar la clave pública del servidor, se registra un error y se finaliza el programa.

2. Bucle principal:

- Se entra en un bucle infinito para escuchar por datos del cliente a través del socket.
- Intenta recibir datos JSON del socket utilizando la función `listen_socket`.
- Si hay un error al escuchar el socket, se registra el error y se continúa en la siguiente iteración del bucle.
- Si se reciben datos JSON, se verifica si contienen la clave `Kill` con valor `True`.
- Si se detecta la señal de apagado, se envía una petición GET a la dirección `http://127.0.0.1:5000/shutdown` para notificar al servidor y se finaliza el programa.

3. Procesamiento de datos del problema:

- Si no se ha recibido la señal de apagado, se verifica si los datos JSON contienen la clave `Problem`.
- Si se encuentra la clave `Problem`, se extraen los valores de `Problem` (tipo de problema), `CountNumbers` (cantidad de números), `MinNumber` (mínimo), `MaxNumber` (máximo), `OutPutFile` (salida a archivo) y `OutPutCmd` (salida por consola).
- Se crea un diccionario con los valores extraídos para enviarlos al servidor.
- Se utiliza la función `encrypt` del objeto `crypter` para cifrar los datos del problema.

4. Envío de datos cifrados al servidor:

- Se envía una petición POST a la dirección `http://127.0.0.1:5000/numbers` con los datos cifrados en formato JSON utilizando la librería `requests`.
- Se establece un tiempo de espera de 10 segundos para la petición.

- Si la petición falla, se registra un error y se continúa en la siguiente iteración del bucle.
- Si la petición es exitosa y el código de estado es 201 (creado), se procesa la respuesta del servidor.

5. Descifrado y procesamiento de resultados:

- Se intenta obtener el resultado cifrado del servidor usando la clave `encrypt_result` de la respuesta JSON.
- Si hay algún error al obtener o procesar la respuesta del servidor, se registra un error y se continúa en la siguiente iteración del bucle.
- Se utiliza la función `decrypt` del objeto `crypter` para descifrar el resultado obtenido del servidor.
- Se extrae la lista de números del resultado descifrado.

6. Solución del problema:

- Se verifica si se han obtenido números del servidor.
- Si hay números, se crea un solucionador de problemas basado en el tipo de problema indicado en los datos recibidos.
- Se usan las clases `CreatorFizzBuzz`, `CreatorFibonacciVerifier` y `CreatorPrimeClassifier` del patrón Fábrica para crear objetos solucionadores específicos según el tipo de problema.
- Se llama al método `solve_problem` del solucionador para obtener los resultados correspondientes a los números recibidos.

7. Envío de resultados al cliente:

- Se crea un diccionario con los datos del problema, los números, los resultados, la opción de salida a archivo y la opción de salida por consola.
- Se utiliza la función `send_to_cli` (no mostrada) para enviar estos datos al cliente por el socket.
- Si hay un error al enviar los resultados al cliente, se registra un error.

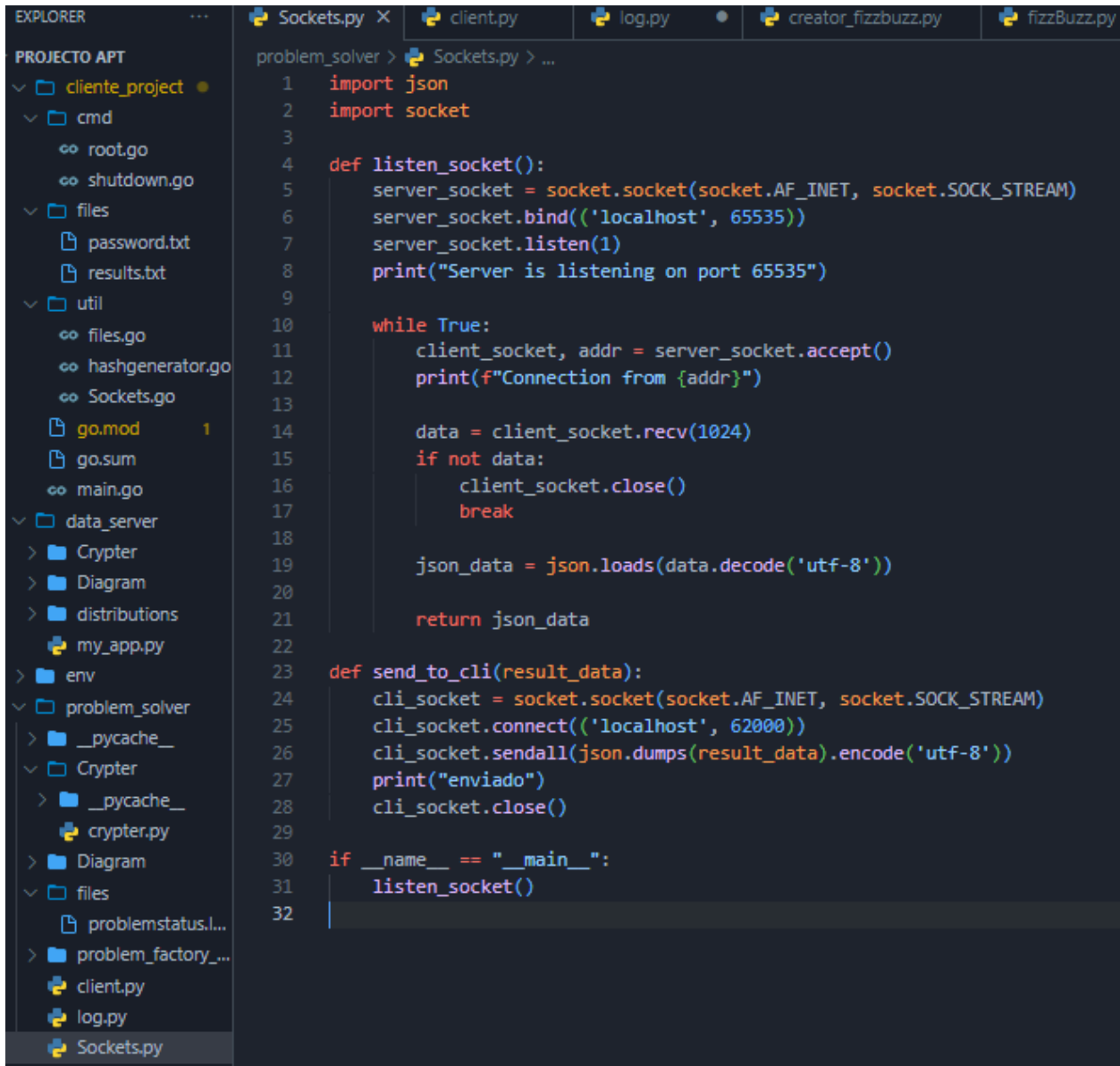
8. Manejo de errores:

- La función `process_and_send_data` se envuelve en un bloque `try-except` para capturar cualquier excepción no manejada.
- Si se produce una excepción, se registra un mensaje crítico con la información del error y se finaliza el programa utilizando `sys.exit()`.

9. Punto de entrada:

- La sección `if __name__ == "__main__":` define el punto de entrada del programa.
- Se llama a la función `process_and_send_data` para iniciar el procesamiento y envío de datos.
- Si se produce una excepción durante la ejecución de la función, se captura y se maneja como se describe en el punto anterior.

Sockets.py



```

1  import json
2  import socket
3
4  def listen_socket():
5      server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
6      server_socket.bind(('localhost', 65535))
7      server_socket.listen(1)
8      print("Server is listening on port 65535")
9
10     while True:
11         client_socket, addr = server_socket.accept()
12         print(f"Connection from {addr}")
13
14         data = client_socket.recv(1024)
15         if not data:
16             client_socket.close()
17             break
18
19         json_data = json.loads(data.decode('utf-8'))
20
21         return json_data
22
23     def send_to_cli(result_data):
24         cli_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
25         cli_socket.connect(('localhost', 62000))
26         cli_socket.sendall(json.dumps(result_data).encode('utf-8'))
27         print("enviado")
28         cli_socket.close()
29
30     if __name__ == "__main__":
31         listen_socket()
32

```

1. Crea un socket de servidor:

```
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

- `socket.socket`: Crea un objeto socket nuevo.
- `socket.AF_INET`: Indica que se usará el protocolo de red IPv4.
- `socket.SOCK_STREAM`: Indica que se usará un socket TCP para comunicación orientada a flujo.

2. Configura el socket del servidor:

- `server_socket.bind(('localhost', 65535))`
 - `bind`: Asocia el socket a una dirección IP y puerto específico.
 - `'localhost'`: Indica que el servidor escuchará en la interfaz de red local (`localhost`).
 - `65535`: Define el puerto en el que el servidor estará escuchando (en este caso, `65535`).

3. Pone el socket a escuchar:

- `server_socket.listen(1)`
 - `listen`: Pone el socket en modo escucha, esperando conexiones entrantes.
 - `1`: Indica la cantidad máxima de conexiones pendientes en la cola de espera.

4. Imprime un mensaje:

- `print("Server is listening on port 65535")`
 - Informa que el servidor está listo para recibir conexiones en el puerto especificado.

5. Bucle infinito para aceptar conexiones:

- `while True:`
 - Inicia un bucle que se repetirá indefinidamente hasta que se detenga el programa.

6. Acepta una conexión:

- `client_socket, addr = server_socket.accept()`
 - `accept`: Acepta una conexión entrante de un cliente y devuelve dos valores:
 - `client_socket`: Un nuevo objeto socket que representa la conexión con el cliente.
 - `addr`: Una tupla que contiene la dirección IP y el puerto del cliente.

7. Imprime información de la conexión:

- `print(f"Connection from {addr}")`
 - Muestra un mensaje indicando que se ha establecido una conexión con un cliente, incluyendo su dirección IP y puerto.

8. Recibe datos del cliente:

- `data = client_socket.recv(1024)`
 - `recv`: Recibe datos del cliente a través del socket.
 - 1024: Define el tamaño máximo de datos a recibir en bytes (en este caso, 1024 bytes).

9. Comprueba si se recibieron datos:

- `if not data:`
 - Si no se recibieron datos (`data` es una cadena vacía), significa que el cliente se desconectó o no envió nada.

10. Cierra el socket del cliente:

- `client_socket.close()`
 - Cierra la conexión con el cliente.

11. Termina el bucle:

- `break`
 - Si no se recibieron datos, sale del bucle principal.

12. Decodifica y convierte los datos a JSON:

- `json_data = json.loads(data.decode('utf-8'))`
 - `decode('utf-8')`: Decodifica los datos recibidos (bytes) a una cadena de texto usando la codificación UTF-8.
 - `json.loads`: Convierte la cadena de texto decodificada a un objeto de datos JSON.

13. Retorna los datos JSON:

- `return json_data`
 - Devuelve el objeto JSON con los datos recibidos del cliente.

Factory_Method/Todos sobre la solución del problema (FizzBuzz, PrimeClassifier, FibonacciVerifier)

```
problem_solver > problem_factory_method > creator_fibonacciVerifier.py > CreatorFibonacciVerifier > factory_method
1  from problem_factory_method.i_problem_creator import IProblemCreator
2  from problem_factory_method.i_problem_solver import IProblemSolver
3  from problem_factory_method.fibonacciVerifier import FibonacciVerifier
4
5  class CreatorFibonacciVerifier(IProblemCreator):
6
7      def factory_method(self) -> IProblemSolver:
8          return FibonacciVerifier()
```

```
problem_solver > problem_factory_method > creator_fizzbuzz.py > CreatorFizzBuzz > factory_method
1  from problem_factory_method.i_problem_creator import IProblemCreator
2  from problem_factory_method.i_problem_solver import IProblemSolver
3  from problem_factory_method.fizzBuzz import FizzBuzz
4
5  class CreatorFizzBuzz(IProblemCreator):
6
7      def factory_method(self) -> IProblemSolver:
8          return FizzBuzz()
```

```
problem_solver > problem_factory_method > creator_primeClassifier.py > CreatorPrimeClassifier > factory_method
1  from problem_factory_method.i_problem_creator import IProblemCreator
2  from problem_factory_method.i_problem_solver import IProblemSolver
3  from problem_factory_method.primeClassifier import PrimeClassifier
4
5  class CreatorPrimeClassifier(IProblemCreator):
6
7      def factory_method(self) -> IProblemSolver:
8          return PrimeClassifier()
```

```
problem_solver > problem_factory_method > fibonacciVerifier.py > FibonacciVerifier > is_fibonacci
1  from problem_factory_method.i_problem_solver import IProblemSolver
2
3  class FibonacciVerifier(IProblemSolver):
4
5      def solve_problem(self, data):
6          result = []
7
8          for number in data:
9
10             line = f"{self.is_fibonacci(number)}"
11
12             result.append(line)
13
14         return result
15
16     def is_fibonacci(self, number):
17
18         if number == 1 or number == 0:
19             return True
20
21         numer_1 = 0
22         numer_2 = 1
23
24         is_fibonacci = False
25
26         actual_fibonacci_number = 0
27
28         while actual_fibonacci_number <= number:
29
30             if actual_fibonacci_number == number:
31                 is_fibonacci = True
32
33             actual_fibonacci_number = numer_1 + numer_2
34
35             numer_1 = numer_2
36
37             numer_2 = actual_fibonacci_number
38
39         return is_fibonacci
```

```
problem_solver > problem_factory_method > fibonacciVerifier.py > FibonacciVerifier > is_fibonacci
1  from problem_factory_method.i_problem_solver import IProblemSolver
2
3  class FibonacciVerifier(IProblemSolver):
4
5      def solve_problem(self, data):
6          result = []
7
8          for number in data:
9
10             line = f"{self.is_fibonacci(number)}"
11
12             result.append(line)
13
14         return result
15
16     def is_fibonacci(self, number):
17
18         if number == 1 or number == 0:
19             return True
20
21         numer_1 = 0
22         numer_2 = 1
23
24         is_fibonacci = False
25
26         actual_fibonacci_number = 0
27
28         while actual_fibonacci_number <= number:
29
30             if actual_fibonacci_number == number:
31                 is_fibonacci = True
32
33             actual_fibonacci_number = numer_1 + numer_2
34
35             numer_1 = numer_2
36
37             numer_2 = actual_fibonacci_number
38
39         return is_fibonacci
```

EXPLORER

PROJECT...
cliente_project
cmd
root.go
shutdown.go
files
password.txt
results.txt
util
files.go
hashgenerator.go
Sockets.go
go.mod
go.sum
main.go
data_server
Crypter
Diagram
distributions
my_app.py
env
problem_solver
__pycache__
Crypter
__pycache__
crypter.py
Diagram
files
problemstatus.l...
problem_factory_...
__pycache__
creator_fibonac...
creator_fizzbuzz...
creator_primeCl...
fibonacciVerfier...
fizzBuzz.py
i_problem_creat...
i_problem_solve...
primeClassifier.py
client.py
log.py
Sockets.py

distribution_solver.py
normal_distribution.py
uniform_distributuion.py
diagSecuenciaData...

problem_solver > problem_factory_method > primeClassifier.py > PrimeClassifier > is_semiprime
1 from problem_factory_method.i_problem_solver import IProblemSolver
2
3 class PrimeClassifier(IProblemSolver):
4
5 def solve_problem(self, data):
6 list_result = []
7 for num in data:
8 result = self.prime_classifier(int(num))
9 list_result.append(str(result))
10
11 return list_result
12
13 def prime_classifier(self, number):
14 if self.is_prime(number):
15 return "Prime"
16 if self.is_cuadratic_semiprime(number):
17 return "Cuadratic Semiprime"
18 if self.is_semiprime(number):
19 return "Semiprime"
20 return number
21
22 def is_prime(self, number):
23 count = 0
24 for i in range(2, number + 1):
25
26 if (number % i) == 0:
27 count += 1
28
29 if count < 2:
30 return True
31
32 return False
33
34 def is_semiprime(self, number):
35 count = 0
36 divisor = []
37 for i in range(2, number):
38 if (number % i) == 0 and self.is_prime(i):
39 divisor.append(i)
40 count += 1
41
42 if count == 2 and divisor[0] * divisor[1] == number:
43
44 return True
45
46 return False
47
48 def is_cuadratic_semiprime(self, number):
49 for i in range(2, number + 1):
50 if (number % i) == 0 and self.is_prime(i):
51 if (i*i) == number:
52 return True
53
54 return False

```

problem_solver > problem_factory_method > i_problem_creator.py > IProblemCreator > solve_problem
1  from abc import ABC, abstractmethod
2  from problem_factory_method.i_problem_solver import IProblemSolver
3
4  class IProblemCreator(ABC):
5
6      @abstractmethod
7      def factory_method(self) -> IProblemSolver:
8          pass
9
10     def solve_problem(self, data) -> list:
11         problem = self.factory_method()
12         result = problem.solve_problem(data)
13     return result

```

```

problem_solver > problem_factory_method > i_problem_solver.py > IProblemSolver > solve_problem
1  from abc import ABC, abstractmethod
2
3  class IProblemSolver(ABC):
4
5      @abstractmethod
6      def solve_problem(self, data: list) -> list:
7          pass

```

1. Interfaces:

- **IProblemSolver:** Define una interfaz base para los solucionadores de problemas.
 - `solve_problem(self, data: list) -> list`: Este método abstracto obliga a las clases que implementen esta interfaz a definir una función `solve_problem` que reciba una lista de datos y retorne una lista con los resultados.
- **IProblemCreator:** Define una interfaz base para los creadores de solucionadores.
 - `factory_method(self) -> IProblemSolver`: Este método abstracto obliga a las clases que implementen esta interfaz a definir una función `factory_method` que cree y retorne un objeto de tipo `IProblemSolver`.

2. Clases concretas de solucionadores:

- **FibonacciVerifier:** Esta clase implementa la interfaz `IProblemSolver`.
 - `solve_problem(self, data: list) -> list`:
 - Recorre la lista de datos (números).
 - Para cada número, llama a la función `is_fibonacci` para verificar si es un número de Fibonacci.
 - Almacena el resultado (`True/False`) en una lista.
 - Retorna la lista con los resultados para todos los números.

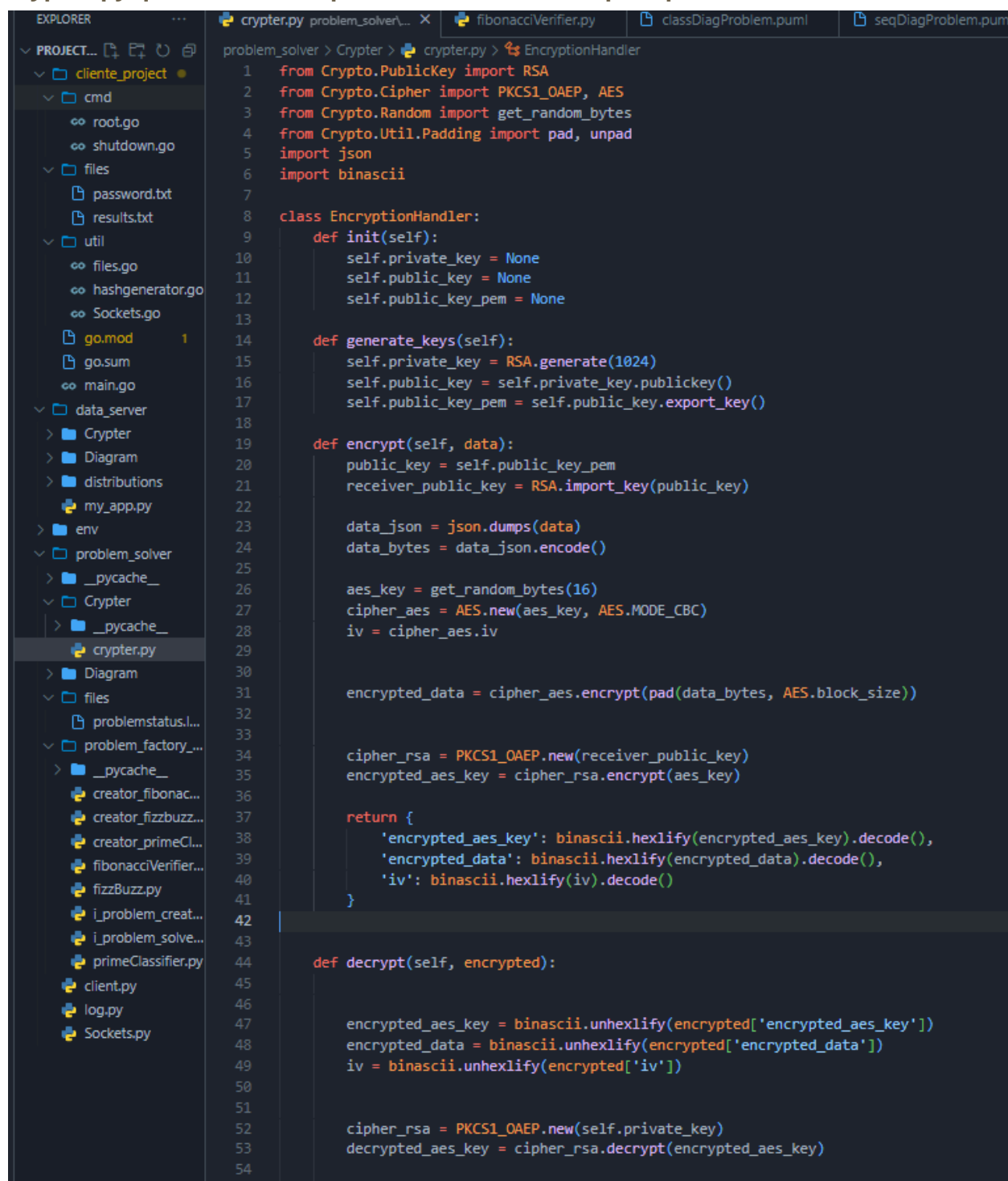
- `is_fibonacci(self, number: int) -> bool:`
 - Verifica si el número es 0 o 1 (casos base de la secuencia de Fibonacci).
 - Si no es un caso base, define variables para dos números anteriores de la secuencia.
 - Utiliza un bucle while para iterar hasta que el número actual de la secuencia sea mayor o igual al número a verificar.
 - Compara el número actual con el número a verificar.
 - Si son iguales, devuelve True (es Fibonacci).
 - Calcula el siguiente número de la secuencia y actualiza las variables para los dos anteriores.
 - Retorna False si no se encuentra el número en la secuencia.
- FizzBuzz: Esta clase implementa la interfaz `IProblemSolver`.
 - `solve_problem(self, data: list) -> list:`
 - Recorre la lista de datos (números).
 - Para cada elemento (número convertido a entero), llama a la función privada `__fizz_buzz` para obtener el resultado.
 - Almacena el resultado en una lista.
 - Retorna la lista con los resultados para todos los números.
 - `__fizz_buzz(self, number: int) -> str:`
 - Define una cadena vacía para el resultado.
 - Verifica si el número es divisible por 3 y agrega "Fizz" al resultado si lo es.
 - Verifica si el número es divisible por 5 y agrega "Buzz" al resultado si lo es.
 - Si el número no es divisible por 3 ni por 5, convierte el número a cadena y lo agrega al resultado.
 - Retorna la cadena resultante (que puede ser "Fizz", "Buzz", "FizzBuzz" o el número convertido a cadena).
- PrimeClassifier: Esta clase implementa la interfaz `IProblemSolver`.
 - `solve_problem(self, data: list) -> list:`
 - Recorre la lista de datos (números).
 - Para cada número, llama a la función `prime_classifier` para clasificar el tipo de número primo.
 - Almacena la clasificación en una lista.
 - Retorna la lista con las clasificaciones para todos los números.

2. Clases concretas de creadores:

- CreatorFibonacciVerifier: Esta clase implementa la interfaz `IProblemCreator`.
 - `factory_method(self) -> IProblemSolver:`

- Crea y retorna un objeto de la clase FibonacciVerifier.
- CreatorFizzBuzz: Esta clase implementa la interfaz IProblemCreator.
 - factory_method(self) -> IProblemSolver:
 - Crea y retorna un objeto de la clase FizzBuzz.
- CreatorPrimeClassifier: Esta clase implementa la interfaz IProblemCreator.
 - factory_method(self) -> IProblemSolver:
 - Crea y retorna un objeto de la clase PrimeClassifier.

Crypter.py que es el mismo para data_server tanto para problem_solver



```

1  from Crypto.PublicKey import RSA
2  from Crypto.Cipher import PKCS1_OAEP, AES
3  from Crypto.Random import get_random_bytes
4  from Crypto.Util.Padding import pad, unpad
5  import json
6  import binascii
7
8  class EncryptionHandler:
9      def init(self):
10         self.private_key = None
11         self.public_key = None
12         self.public_key_pem = None
13
14     def generate_keys(self):
15         self.private_key = RSA.generate(1024)
16         self.public_key = self.private_key.publickey()
17         self.public_key_pem = self.public_key.export_key()
18
19     def encrypt(self, data):
20         public_key = self.public_key_pem
21         receiver_public_key = RSA.import_key(public_key)
22
23         data_json = json.dumps(data)
24         data_bytes = data_json.encode()
25
26         aes_key = get_random_bytes(16)
27         cipher_aes = AES.new(aes_key, AES.MODE_CBC)
28         iv = cipher_aes.iv
29
30         encrypted_data = cipher_aes.encrypt(pad(data_bytes, AES.block_size))
31
32         cipher_rsa = PKCS1_OAEP.new(receiver_public_key)
33         encrypted_aes_key = cipher_rsa.encrypt(aes_key)
34
35         return {
36             'encrypted_aes_key': binascii.hexlify(encrypted_aes_key).decode(),
37             'encrypted_data': binascii.hexlify(encrypted_data).decode(),
38             'iv': binascii.hexlify(iv).decode()
39         }
40
41     def decrypt(self, encrypted):
42
43         encrypted_aes_key = binascii.unhexlify(encrypted['encrypted_aes_key'])
44         encrypted_data = binascii.unhexlify(encrypted['encrypted_data'])
45         iv = binascii.unhexlify(encrypted['iv'])
46
47         cipher_rsa = PKCS1_OAEP.new(self.private_key)
48         decrypted_aes_key = cipher_rsa.decrypt(encrypted_aes_key)
49
50
51
52
53
54

```

```

33
34 cipher_rsa = PKCS1_OAEP.new(receiver_public_key)
35 encrypted_aes_key = cipher_rsa.encrypt(aes_key)
36
37
38 return {
39     'encrypted_aes_key': binascii.hexlify(encrypted_aes_key).decode(),
40     'encrypted_data': binascii.hexlify(encrypted_data).decode(),
41     'iv': binascii.hexlify(iv).decode()
42 }
43
44
45 def decrypt(self, encrypted):
46
47     encrypted_aes_key = binascii.unhexlify(encrypted['encrypted_aes_key'])
48     encrypted_data = binascii.unhexlify(encrypted['encrypted_data'])
49     iv = binascii.unhexlify(encrypted['iv'])
50
51
52     cipher_rsa = PKCS1_OAEP.new(self.private_key)
53     decrypted_aes_key = cipher_rsa.decrypt(encrypted_aes_key)
54
55     cipher_aes = AES.new(decrypted_aes_key, AES.MODE_CBC, iv)
56     decrypted_data = unpad(cipher_aes.decrypt(encrypted_data), AES.block_size)
57
58     decrypted_json = decrypted_data.decode()
59
60     return json.loads(decrypted_json)

```

1. init(self):

- Inicializa los atributos de la clase:
 - private_key: Clave privada RSA (inicialmente None).
 - public_key: Clave pública RSA (inicialmente None).
 - public_key_pem: Clave pública RSA en formato PEM (inicialmente None).

2. generate_keys(self):

- Genera un par de claves RSA aleatorias de 1024 bits.
- Almacena la clave privada en self.private_key.
- Almacena la clave pública en self.public_key.
- Exporta la clave pública a formato PEM y la almacena en self.public_key_pem.

3. encrypt(self, data):

- Esta función cifra los datos utilizando una combinación de RSA y AES.
 - Recupera la clave pública del receptor en formato PEM desde self.public_key_pem.
 - Importa la clave pública del receptor a un objeto RSA.
 - Convierte los datos a un objeto JSON y luego a bytes.
 - Genera una clave aleatoria AES de 16 bytes.
 - Crea un objeto de cifrado AES en modo CBC con la clave aleatoria.
 - Obtiene el vector de inicialización (IV) del objeto AES.
 - Utiliza padding PKCS#7 para rellenar los datos al tamaño del bloque AES.

- Cifra los datos con el objeto AES.
- Crea un objeto de cifrado PKCS1_OAEP utilizando la clave pública del receptor.
- Cifra la clave AES aleatoria con el objeto de cifrado PKCS1_OAEP.
- La función retorna un diccionario con los siguientes elementos:
 - encrypted_aes_key: Clave AES cifrada en formato hexadecimal.
 - encrypted_data: Datos cifrados con AES en formato hexadecimal.
 - iv: Vector de inicialización en formato hexadecimal.

4. decrypt(self, encrypted):

- Esta función descifra los datos previamente cifrados con la función encrypt.
 - Recupera la clave AES cifrada, los datos cifrados y el vector de inicialización del diccionario encrypted.
 - Convierte los elementos del diccionario de hexadecimal a bytes.
 - Crea un objeto de cifrado PKCS1_OAEP utilizando la clave privada del objeto.
 - Descifra la clave AES aleatoria utilizando el objeto de cifrado PKCS1_OAEP.
 - Crea un objeto de descifrado AES en modo CBC con la clave AES descifrada y el vector de inicialización.
 - Descifra los datos con el objeto AES.
 - Elimina el padding PKCS#7 de los datos descifrados.
 - Convierte los datos descifrados a un objeto JSON y luego a una cadena.
- La función retorna el objeto JSON descifrado.

data_server/my_app.py

```

EXPLORER
PROJECT APT
  cliente_project
    cmd
    root.go
    shutdown.go
    files
    password.txt
    results.txt
    util
    files.go
    hashgenerator.go
    Sockets.go
    go.mod
    go.sum
    main.go
  data_server
    Crypter
    Diagram
    distributions
    my_app.py
  env
  problem_solver
  _pycache_
  Crypter
  _pycache_
  crypter.py
  Diagram
  files
  problemstatus.l...
  problem_factory_...
  _pycache_
  creator_fibonac...
  creator_fizzbuzz...
  creator_primeCl...
  fibonacciVerifier...
  fizzBuzz.py
  i_problem_creat...
  i_problem_solve...
  primeClassifier.py
  client.py
  log.py
  Sockets.py

data_server > my_app.py > MyApp > __init__ > generate_numbers
1 from flask import Flask, request, jsonify
2 from distributions.normal_distribution import NormalDistribution
3 from distributions.uniform_distributuion import UniformDistribution
4 from Crypter.crypter import EncryptionHandler
5 import random, os, signal
6
7 class MyApp:
8     def __init__(self):
9         self.app = Flask(__name__)
10        self.crypter = EncryptionHandler()
11        self.crypter.generate_keys()
12
13    @self.app.route('/public_key', methods=['POST'])
14    def public_keys_change():
15        data = request.get_json()
16        if 'public_key' in data:
17            public_key = self.crypter.public_key_pem.decode()
18            self.crypter.public_key_pem = data['public_key'].encode()
19
20        data = {'public_key': public_key}
21        return jsonify(data), 200
22
23    @self.app.route('/numbers', methods=['POST'])
24    def generate_numbers():
25        data = request.get_json()
26
27        if not data:
28            return jsonify({'error': 'no data'})
29        try:
30            decrypt_data = self.crypter.decrypt(data)
31        except ValueError as e:
32            return jsonify({'error': 'Decryption failed'}), 500
33
34        min = int(decrypt_data['min'])
35        max = int(decrypt_data['max'])
36        count = int(decrypt_data['count'])
37
38        option = random.choice(['normal', 'uniform'])
39        if option == 'normal':
40            numbers, distribution = NormalDistribution().generate_numbers(min, max, count)
41
42        if option == 'uniform':
43            numbers, distribution = UniformDistribution().generate_numbers(min, max, count)
44
45        result = {
46            'numbers': numbers,
47            'distribution': distribution
48        }
49        try:
50            encrypt_result = self.crypter.encrypt(result)
51            return jsonify({'encrypt_result': encrypt_result}), 201
52        except ValueError as e:
53            return jsonify({'error': 'Encryption failed', 'message': str(e)}), 500
54

```

```

40         numbers, distribution = NormalDistribution().generate_numbers(min, max, count)
41
42     if option == 'uniform':
43         numbers, distribution = UniformDistribution().generate_numbers(min, max, count)
44
45     result = {
46         'numbers': numbers,
47         'distribution': distribution
48     }
49     try:
50         encrypt_result = self.crypter.encrypt(result)
51         return jsonify({'encrypt_result': encrypt_result}), 201
52     except ValueError as e:
53         return jsonify({'error': 'Encryption failed', 'message': str(e)}), 500
54
55 @self.app.route('/shutdown', methods=['GET'])
56 def shutdown():
57     print("Shutting down the server flask")
58     pid = os.getpid()
59     os.kill(pid, signal.SIGINT)
60
61
62 def run_app(self):
63     self.app.run(host='0.0.0.0', port=5000, debug=True)
64
65 if __name__ == '__main__':
66     app = MyApp()
67     app.run_app()
68

```

1. Atributos:

- `self.app`: Representa la instancia de la aplicación Flask.
- `self.crypter`: Representa una instancia de la clase `EncryptionHandler` utilizada para el cifrado y descifrado de datos.

2. Constructor (`__init__`):

- Inicializa la aplicación Flask.
- Crea una instancia de `EncryptionHandler` para manejar el cifrado.
- Genera un par de claves RSA y las almacena en el objeto `crypter`.

3. Rutas de la aplicación:

- **/public_key (POST):**
 - Esta ruta permite al cliente enviar su clave pública al servidor.
 - Recupera los datos JSON enviados en la petición.
 - Si los datos contienen la clave pública (`public_key`), reemplaza la clave pública actual almacenada en el servidor por la recibida del cliente.
 - Devuelve la clave pública del servidor en formato JSON.
- **/numbers (POST):**
 - Esta ruta permite al cliente solicitar la generación de números aleatorios.
 - Recupera los datos JSON enviados en la petición.

- Si no hay datos en la petición, devuelve un error indicando que no hay datos.
- Intenta descifrar los datos recibidos utilizando el objeto crypter.
- En caso de error en el descifrado, devuelve un error indicando "Fallo en la descriptación".
- Si el descifrado es exitoso, extrae los valores mínimos, máximos y cantidad de números a generar de los datos descifrados.
- Elige aleatoriamente entre generar una distribución normal o uniforme.
- Si se elige la normal, utiliza la clase NormalDistribution para generar los números y la distribución correspondiente.
- Si se elige la uniforme, utiliza la clase UniformDistribution para generar los números y la distribución correspondiente.
- Crea un diccionario con los resultados que incluye:
 - numbers: Lista de números aleatorios generados.
 - distribution: Tipo de distribución utilizada (normal o uniforme).
- Intenta cifrar el diccionario de resultados utilizando el objeto crypter.
- En caso de error en el cifrado, devuelve un error indicando "Fallo en la encriptación".
- Si el cifrado es exitoso, devuelve el resultado cifrado en formato JSON.
- **/shutdown (GET):**
 - Esta ruta permite apagar el servidor Flask.
 - Imprime un mensaje indicando que se está apagando el servidor.
 - Obtiene el identificador del proceso actual.
 - Envía una señal de interrupción (SIGINT) al proceso actual para detenerlo.

4. Método run_app:

- Inicia la ejecución de la aplicación Flask.
- Especifica la dirección IP (0.0.0.0) para que la aplicación sea accesible desde cualquier dispositivo en la red.
- Establece el puerto en 5000 (port=5000).
- Activa el modo debug (debug=True) para facilitar la depuración durante el desarrollo.

distribuciones en data_server (Normal distribution y uniform distribution

```

data_server > distributions > i_distribution_solver.py > ...
1  from abc import ABC, abstractmethod
2
3  class IDistributionRandomSolver(ABC):
4
5      @abstractmethod
6      def generate_numbers(self, min, max, count):
7          pass
8
9

```

```

data_server > distributions > normal_distribution.py > ...
1  import random
2  from distributions.i_distribution_solver import IDistributionRandomSolver
3
4  class NormalDistribution(IDistributionRandomSolver):
5
6      def generate_numbers(self, min, max, count):
7          media = (min+max)/2
8          sigma = (max - min)/3
9          numbers = [round(random.gauss(media, sigma)) for _ in range(count)]
10         distribution = 'normal'
11
12         return numbers, distribution
13
14
15

```

```

data_server > distributions > uniform_distribution.py > UniformDistribution > generate_numbers
1  import random
2  from distributions.i_distribution_solver import IDistributionRandomSolver
3
4  class UniformDistribution(IDistributionRandomSolver):
5      def generate_numbers(self, min, max, count):
6          numbers = [round(random.uniform(min, max)) for _ in range(count)]
7          distribution = 'uniform'
8
9      return numbers, distribution

```

1. Interfaz IDistributionRandomSolver (ABC):

- Esta interfaz define un método abstracto `generate_numbers(self, min, max, count)`.
- Este método obliga a las clases que implementen esta interfaz a definir una función `generate_numbers` que reciba tres parámetros:
 - `min`: Valor mínimo del rango para generar los números.
 - `max`: Valor máximo del rango para generar los números.

- `count`: Cantidad de números aleatorios a generar.
- La función `generate_numbers` debe retornar una tupla con dos elementos:
 - Primer elemento: Lista de números aleatorios generados.
 - Segundo elemento: Tipo de distribución utilizada (cadena de texto).

2. Clase `NormalDistribution`:

- Esta clase implementa la interfaz `IDistributionRandomSolver`.
- El método `generate_numbers` implementa la lógica para generar números aleatorios con distribución normal.
 - Calcula la media como el promedio del mínimo y máximo del rango.
 - Calcula la desviación estándar (`sigma`) como un tercio de la diferencia entre el mínimo y el máximo.
 - Utiliza la función `random.gauss(media, sigma)` para generar números aleatorios con distribución normal `count` veces.
 - Almacena los números generados en una lista.
 - Define el tipo de distribución como "normal".
- La función retorna una tupla con la lista de números generados y el tipo de distribución ("normal").

3. Clase `UniformDistribution`:

- Esta clase implementa la interfaz `IDistributionRandomSolver`.
- El método `generate_numbers` implementa la lógica para generar números aleatorios con distribución uniforme.
- Utiliza la función `random.uniform(min, max)` para generar números aleatorios uniformes dentro del rango especificado `count` veces.
- Almacena los números generados en una lista.
- Define el tipo de distribución como "uniform".
- La función retorna una tupla con la lista de números generados y el tipo de distribución ("uniform").

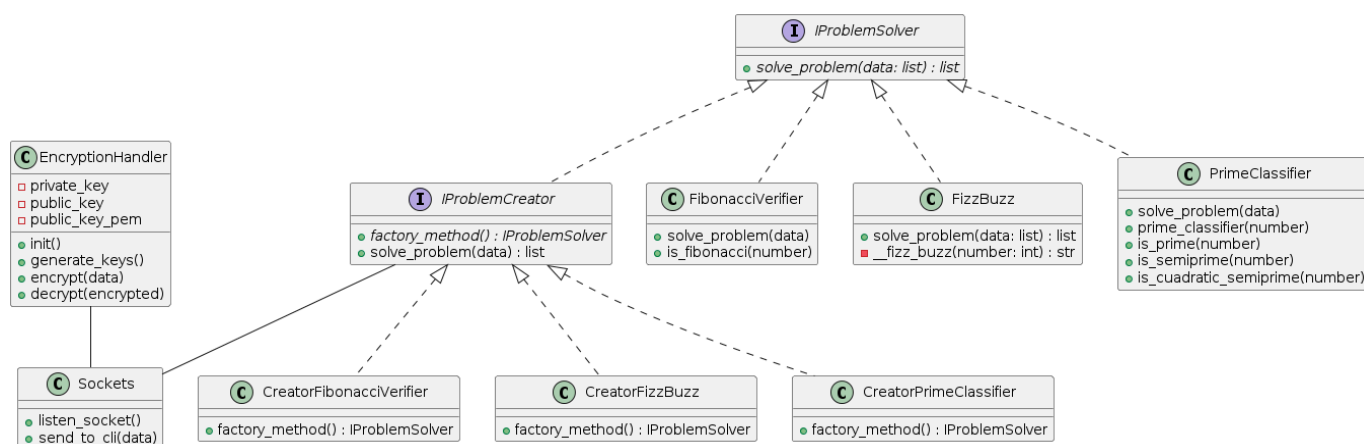
DIAGRAMAS

DIAGRAMA DE CLASES EN PROBLEM SOLVER

Este diagrama de clases modela un sistema que utiliza el Patrón Fábrica para crear objetos solucionadores de problemas matemáticos específicos.

Relaciones:

- Las clases CreatorFibonacciVerifier, CreatorFizzBuzz y CreatorPrimeClassifier implementan la interfaz IProblemCreator.
- Las clases FibonacciVerifier, FizzBuzz y PrimeClassifier implementan la interfaz IProblemSolver.
- La clase EncryptionHandler se relaciona con la clase Sockets (posiblemente para cifrar la comunicación por sockets).
- La clase Sockets se relaciona con las clases IProblemCreator (posiblemente para recibir solicitudes de solución de problemas).



CÓDIGO FUENTE DIAGRAMA DE CLASES

@startuml Diagrama de clases ProblemSolver

```

class EncryptionHandler {
    -private_key
    -public_key

```

```
-public_key_pem
+init()
+generate_keys()
+encrypt(data)
+decrypt(encrypted)
}
```

```
interface IProblemCreator {
    {abstract} +factory_method() : IProblemSolver
    +solve_problem(data) : list
}
```

```
interface IProblemSolver {
    {abstract} +solve_problem(data: list) : list
}
```

```
IProblemCreator <|.. CreatorFibonacciVerifier
IProblemCreator <|.. CreatorFizzBuzz
IProblemCreator <|.. CreatorPrimeClassifier
IProblemSolver <|.. IProblemCreator
```

```
IProblemSolver <|.. FibonacciVerifier
IProblemSolver <|.. FizzBuzz
IProblemSolver <|.. PrimeClassifier
```

```
class CreatorFibonacciVerifier {
    +factory_method() : IProblemSolver
}
```

```
class CreatorFizzBuzz {  
    +factory_method() : IProblemSolver  
}
```

```
class CreatorPrimeClassifier {  
    +factory_method() : IProblemSolver  
}
```

```
class FibonacciVerifier {  
    +solve_problem(data)  
    +is_fibonacci(number)  
}
```

```
class FizzBuzz {  
    +solve_problem(data: list) : list  
    - __fizz_buzz(number: int) : str  
}
```

```
class PrimeClassifier {  
    +solve_problem(data)  
    +prime_classifier(number)  
    +is_prime(number)  
    +is_semiprime(number)  
    +is_cuadratic_semiprime(number)  
}
```

```
class Sockets {  
    +listen_socket()  
    +send_to_cli(data)
```

}

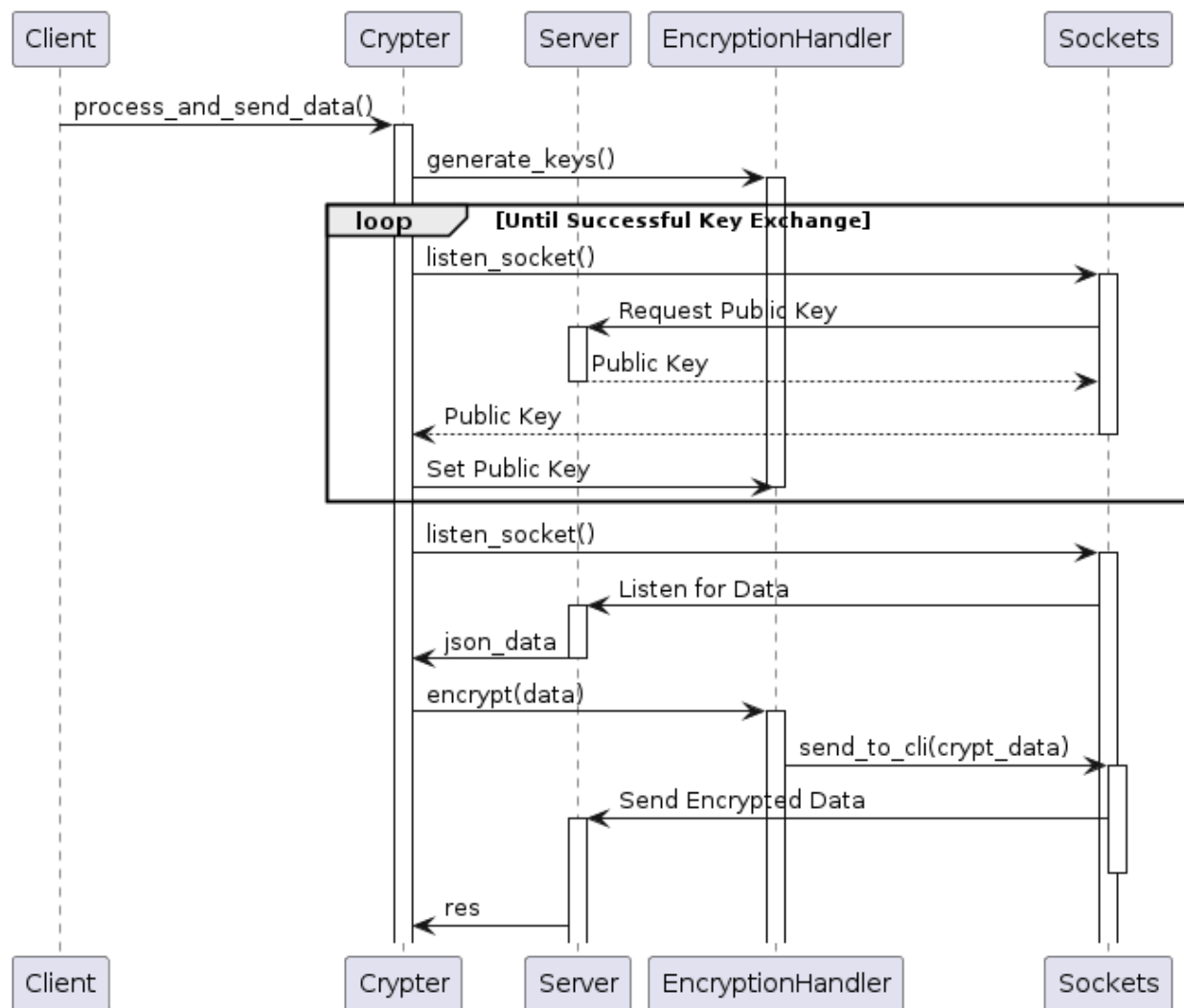
EncryptionHandler -- Sockets

IProblemCreator -- Sockets

@enduml

SECUENCIA PROBLEM SOLVER

Este diagrama de secuencia muestra el flujo de comunicación entre el cliente, el servidor y los componentes involucrados en el cifrado y la transferencia de datos.



Participantes:

- Cliente: Representa la aplicación cliente que solicita datos al servidor.
- Crypter: Componente encargado del cifrado y descifrado de datos.
- Servidor: Representa la aplicación del servidor que procesa las solicitudes del cliente.
- EncryptionHandler: Maneja la generación y uso de claves para el cifrado.
- Sockets: Se encarga de la comunicación por sockets (establece la conexión y transmite datos).

CÓDIGO FUENTE DIAGRAMA DE SECUENCIA PROBLEM SOLVER

@startuml Diagrama De Secuencia ProblemSolver

participant Client

participant Crypter

participant Server

participant EncryptionHandler

participant Sockets

Client -> Crypter: process_and_send_data()

activate Crypter

Crypter -> EncryptionHandler: generate_keys()

activate EncryptionHandler

loop Until Successful Key Exchange

 Crypter -> Sockets: listen_socket()

 activate Sockets

 Sockets -> Server: Request Public Key

 activate Server

 Server --> Sockets: Public Key

 deactivate Server

 Sockets --> Crypter: Public Key

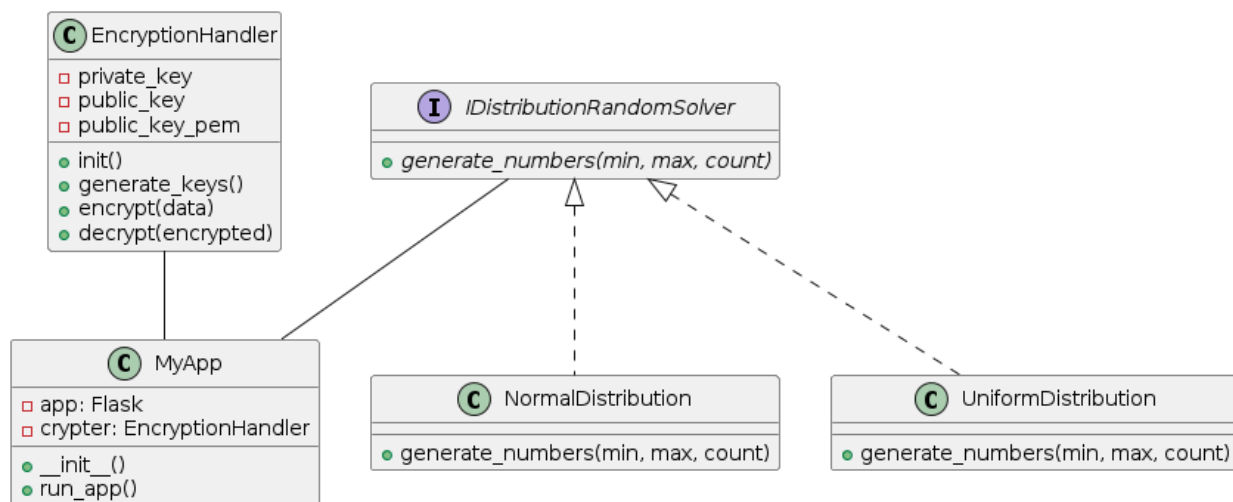
 deactivate Sockets



```
Crypter -> EncryptionHandler: Set Public Key
deactivate EncryptionHandler
end
Crypter -> Sockets: listen_socket()
activate Sockets
Sockets -> Server: Listen for Data
activate Server
Server -> Crypter: json_data
deactivate Server
Crypter -> EncryptionHandler: encrypt(data)
activate EncryptionHandler
EncryptionHandler -> Sockets: send_to_cli(crypt_data)
activate Sockets
Sockets -> Server: Send Encrypted Data
activate Server
deactivate Sockets
Server -> Crypter: res
```

DIAGRAMA DE CLASES EN DATA SERVER

Este diagrama de clases representa la estructura de una aplicación Flask que utiliza cifrado y distribuciones de probabilidad para generar números aleatorios.



Relaciones:

- La clase **MyApp** tiene una instancia de la clase **EncryptionHandler** para el cifrado y descifrado.
- La clase **MyApp** también tiene una relación con la interfaz **IDistributionRandomSolver**. Esto permite a **MyApp** trabajar con diferentes clases que implementen esta interfaz para generar números con distintas distribuciones.
- Las clases **NormalDistribution** y **UniformDistribution** implementan la interfaz **IDistributionRandomSolver**, indicando que pueden generar números aleatorios y comparten el mismo método `generate_numbers`.

CÓDIGO FUENTE DIAGRAMA DE CLASES DATA SERVER

@startuml Diagrama De Clases Jesus Gelves

```

class EncryptionHandler {
    -private_key
    -public_key
    -public_key_pem
    +init()
    +generate_keys()
  
```

```
+encrypt(data)
+decrypt(encrypted)
}

interface IDistributionRandomSolver {
    {abstract} +generate_numbers(min, max, count)
}

class NormalDistribution {
    +generate_numbers(min, max, count)
}

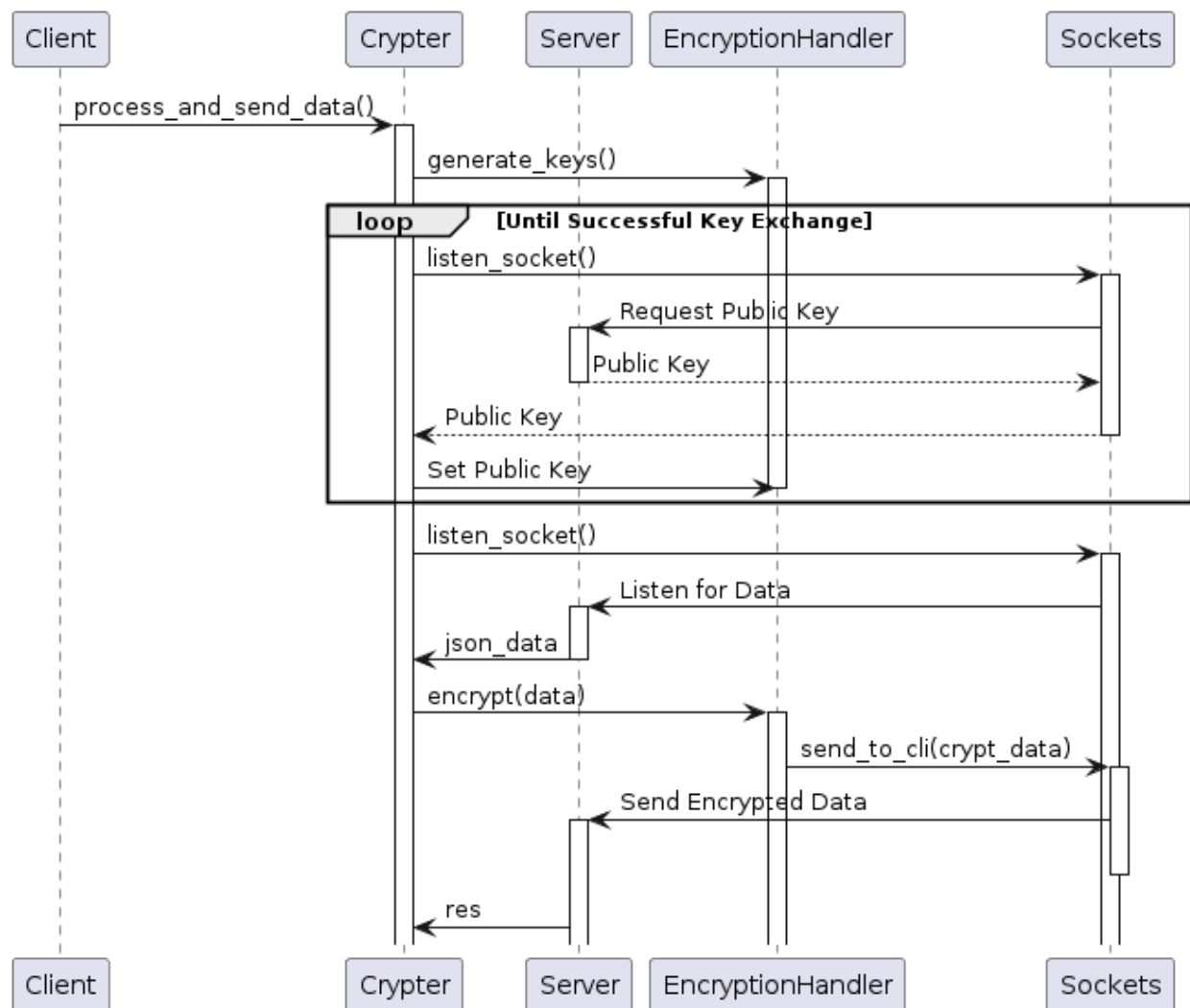
class UniformDistribution {
    +generate_numbers(min, max, count)
}

class MyApp {
    -app: Flask
    -crypter: EncryptionHandler
    +__init__()
    +run_app()
}

IDistributionRandomSolver -- MyApp
EncryptionHandler -- MyApp
IDistributionRandomSolver <|.. NormalDistribution
IDistributionRandomSolver <|.. UniformDistribution
@enduml
```

SECUENCIA DATA SERVER

Este diagrama de secuencia detalla el intercambio de datos entre el cliente, el servidor y los componentes involucrados en la generación y transmisión de números aleatorios cifrados.



Participantes:

- Cliente: Representa la aplicación cliente que solicita números aleatorios.
- Servidor: Representa la aplicación del servidor que procesa las solicitudes del cliente y genera los números aleatorios.
- EncryptionHandler: Maneja la generación y uso de claves para el cifrado.
- Crypter: Componente encargado del cifrado y descifrado de datos.

- `IDistributionRandomSolver`: Interfaz para las clases que generan números aleatorios.
- `NormalDistribution` (ejemplo): Clase concreta que implementa la interfaz para generar números con distribución normal.
- `UniformDistribution` (no mostrada): Otra clase concreta que podría implementar la interfaz para generar números con distribución uniforme.
- `Sockets`: Se encarga de la comunicación por sockets (establece la conexión y transmite datos).

CÓDIGO FUENTE DIAGRAMA DE SECUENCIA DATA SERVER

@startuml Diagrama De Secuencia DataServer

participant Client

participant Server

participant EncryptionHandler

participant Crypter

participant IDistributionRandomSolver

participant NormalDistribution

participant UniformDistribution

participant Sockets

Client -> Server: POST /public_key

Server --> Client: 200 OK

Client -> Server: POST /numbers

Server --> Client: 201 OK with encrypted data

Client -> Crypter: decrypt(encrypted_data)

Crypter --> Client: decrypted_data

Client -> IDistributionRandomSolver: generate_numbers(min, max, count)

IDistributionRandomSolver -> NormalDistribution: generate_numbers(min, max, count)

NormalDistribution --> IDistributionRandomSolver: numbers, distribution

IDistributionRandomSolver --> Client: numbers, distribution

Client -> Crypter: encrypt(numbers, distribution)

Crypter --> Client: encrypted_result



Client -> Server: POST /numbers with encrypted_result

Server --> Client: 201 OK

@enduml

METACOGNICIÓN:

En proceso de desarrollo del proyecto, se pudo afianzar el tema de intercambiar una encriptación de datos para temas de seguridad, el cual asimismo fue una dificultad grande al hacer, interesante pero a su vez estresante al lidiar con cosas nuevas.

En el trabajo de interfaces abstractas se refuerza una vez más comprensión sobre el uso de interfaces utilizando el módulo abc y poder hacer bien la estructura que se pide en la inversión de dependencias, como uso de herramientas externas se hizo uso de inteligencia artificial para solucionar diferentes problemas con el tema de encriptación y el sockets al enviar y recibir datos, fue muy útil la inteligencia al sugerir buenas prácticas, ejemplos de código y ayuda durante el proceso de errores.